Agenda

- sum of every subarray
- Max subarray sum of len = K

Qu A[]

find total sum of all subarrays.

Google
facebook

| 2 | 8 | 1 | 3 |
|---|---|---|---|

subarrays:

2 ——— 2
2 8 ——— 10
2 8 1 ——— 11
2 8 1 3 ——— 14
8 ——— 8
8 1 ——— 9
8 1 3 ——— 12
1 ——— 1
1 3 ——— 4
3 ——— 3

74 Ans

```
totalsum=0;
for(i=0 ——— n-1) {
    for(j=i ——— n-1) {
        subarray [i,j]
        subArraysum=0;
        for(k=i ——— j) {
            subArraysum += A[k];
        }
        totalsum += subArraysum;
    }
}
```

TC: $O(n^3)$
SC: $O(1)$

## Prefix sum

sum[l, r] = pf[r] − pf[l−1]

l == 0 ——— pf[r]

```
pf[] = getPrefixsum(A);

totalsum = 0;
for(i = 0 ——————————— n-1) {
    for(j = i ——————————— n-1) {
        subarray [i, j]
        subArraysum = 0;
        if(i == 0) {
            subArraysum = pf[j];
        } else {
            subArraysum = pf[j] - pf[i-1]
        }
        totalsum += subArraysum;
    }
}
```

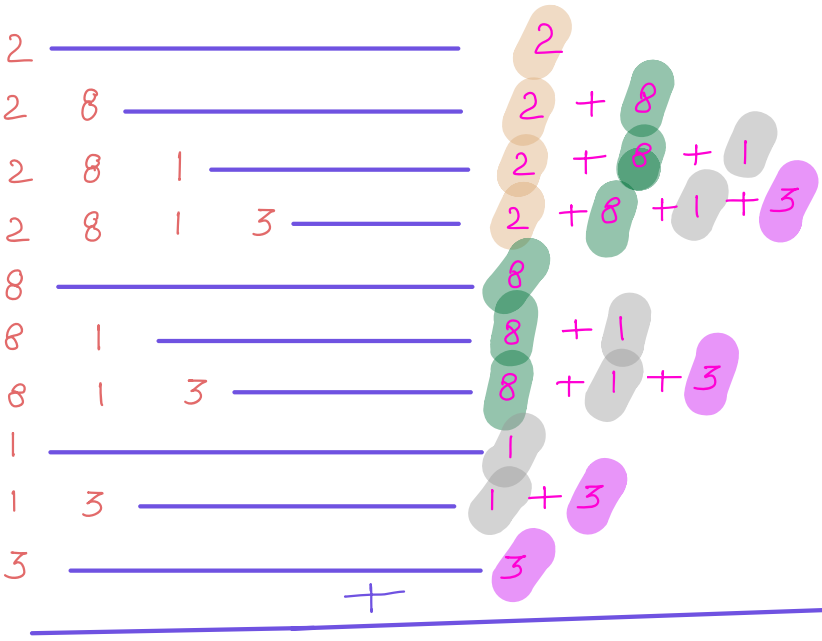
TC: $O(n^2)$

SC: $O(1)$

Expectation : TC: O(n)
SC: O(1)

| 2 | 8 | 1 | 3 |
|---|---|---|---|

Subarrays:




→ 2 * 4 + 8 * 6 + 1 * 6 + 3 * 4.

no of occurences of 2 in all subarrays

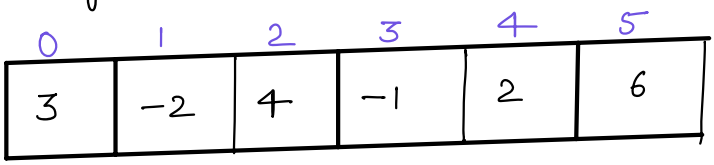
→ $A[0] * occ_0 + A[1] * occ_1 + A[2] * occ_2 + A[3] * occ_3$

Challenge: occurences?

→ $A[0] * [(0+1) * (4-0)] + A[1] * [(1+1) * (4-1)] + A[2] * [(2+1) * (4-2)] + A[3] * [3+1 * 4-3]$

→ A[0] * 4 + A[1] * 6 + A[2] * 6 + A[3] * 4

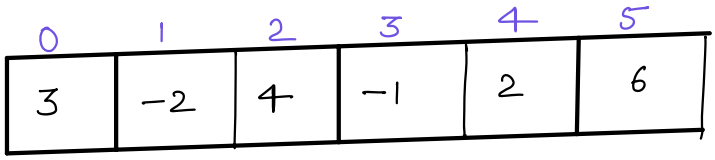
focus: How to find occurances?




occ of idx = 1.

Ans = 10

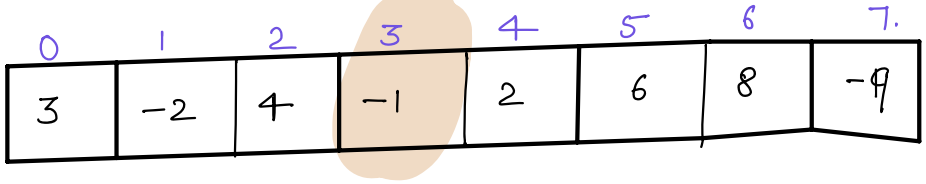
3 -2
3 -2 4
3 -2 4 -1
3 -2 4 -1 2
3 -2 4 -1 2 6
-2
-2 4
-2 4 -1
-2 4 -1 2
-2 4 -1 2 6




occ of idx = 2.

Ans = 12

3 -2 4 -1 2 6
-2 4 -1 2 6
4 4 -1 4 -1 2 4 -1 2 6

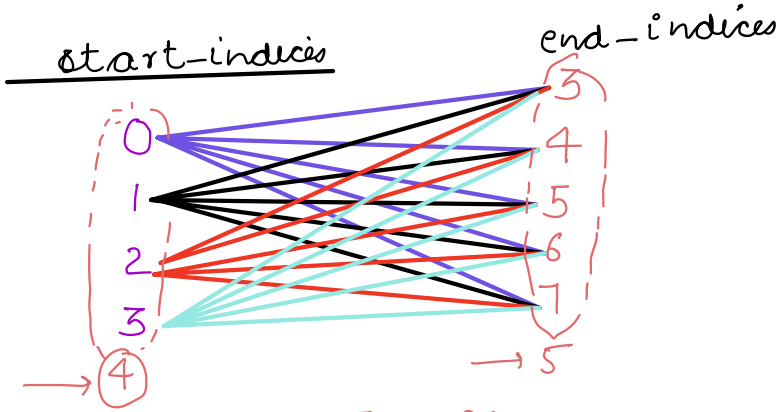



Occ of idx = 3.

$(n-i) * (i+1)$

# of start_indices for which idx=3 is part of ? ⟶ 4. ——— $(i+1)$

# of end_indices for which idx=3 is part of ? ⟶ 5. ——— $(n-i)$




Ans = 4 * 5 = 20

⟶ Occ i ⟶ $(i+1) * (n-i)$

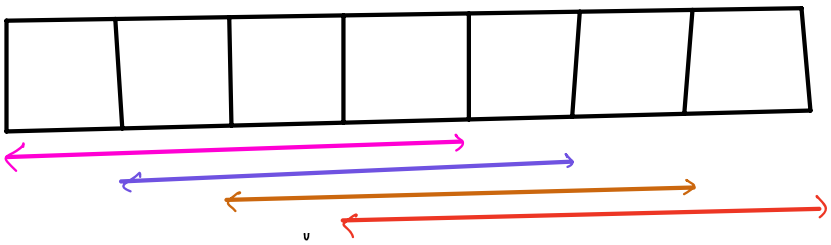
```
total_sum = 0;
for(i=0 ——————— n-1) {
    total_sum += A[i] * {(i+1) * (n-i)}
}
return total_sum;
```

TC: O(n)
SC: O(1)

Break: 10:18 – 10:25

Qu  n = 7

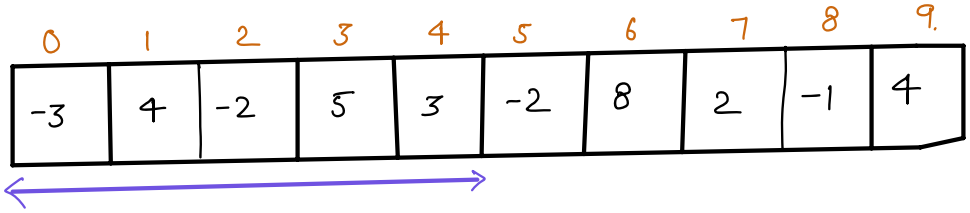
total no of subarrays of len = K ⟶ 4



# of start-indices for which idx=3 is part of ? ⟶ 4.

Qu  A[]  Return max subarray sum with len = K.

Amazon
Uber




K = 5

| s | e | sum |
|---|---|---|
| 0 | 4 | 7 |
| 1 | 5 | 8 |
| 2 | 6 | 12 |
| 3 | 7 | 16 |
| 4 | 8 | 10 |
| 5 | 9 | 11 |

Brute :

```
ans = -∞;
s = 0
e = K-1;
while( e < n ) {
    sum = 0;
    for(i = s ——————— e) {
        sum += A[i];
    }
    ans = max(sum, ans);
    s++;
    e++;
}
return ans;
```

TC: O((n-k) * K)
SC: O(1)

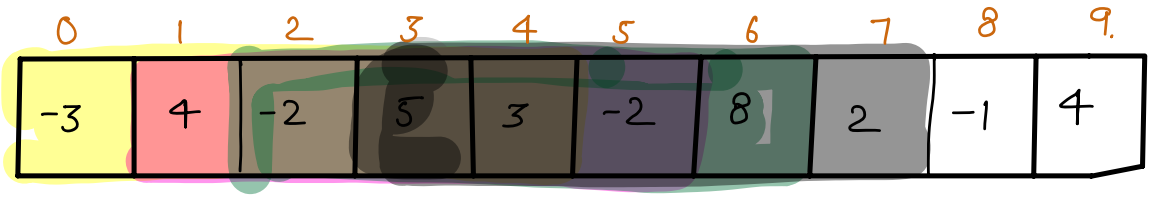
## Prefix :

```
pf[] = getPrefixSum(A);                    ——— n
ans = -∞;
s = 0
e = 0+ k-1;
while( e < n ) {                           ——— n-k
    sum = 0;
    if(s==0) {
        sum = pf[e];
    } else {
        sum = pf[e] - pf[s-1];
    }
    ans = max(sum, ans);
    s++;
    e++;
}
return ans;
```

TC: O(n)
SC: O(n)

## Expectation :

TC: linear
SC: O(1)




K = 5

| s | e | sum |
|---|---|---|
| 0 | 4 | 7 |
| 1 | 5 | 7 - (-3) + (-2) = 8<br>sum - A[0] + A[5] |
| 2 | 6 | 8 - 4 + 8 = 12<br>sum - A[1] + A[6] |
| 3 | 7 | 12 - (-2) + 2 = 16<br>sum - A[2] + A[7] |
| 4 | 8 | |
| 5 | 9 | |

Generalise:

sum - A[s-1] + A[e]

ans = -∞; ⟶ Integer.MIN_VALUE

sum = 0

first window

```
for(i=0; i<=k-1; i++) {
    sum += A[i];
}
s = 1;
e = k;
while(e < n) {
    sum = sum - A[s-1] + A[e];
    ans = max(ans, sum);
    s++;
    e++;
}
return ans;
```

TC: O(n)

SC: O(1)

Next: Memory Management & sorting Basics

→ How memory really work behind scenes?

→ sorting ——— How?
Why?

→ Base: searching. Trees. Graphs

→ Interviews

Thankyou ☺